CodeArena

Competitive Coding Challenge Platform

Phase 1 — Project Design & Documentation

Author	Maghan Das
Neptun code	CCQ946
Date	April 2026

1. Project Overview

1.1 Topic

CodeArena is a full-stack web application that allows users to solve programming challenges and compete in timed coding contests. It combines a persistent challenge library with live competitive features: a real-time leaderboard, timed submissions, and a per-user performance profile.

The platform is inspired by services such as LeetCode and Codeforces, but designed as a self-hostable, community-oriented tool where instructors and admins can author their own challenges and run structured contests for a defined user group.

1.2 Short Description

Users register, browse a library of programming challenges sorted by difficulty, and submit solutions in a browser-based code editor. Each submission is evaluated against pre-defined test cases stored in the database. A verdict (Accepted, Wrong Answer, Time Limit Exceeded) is returned immediately. During an active contest, the leaderboard refreshes every few seconds, showing live rankings based on cumulative score and solve time. Admins manage all challenges, test cases, and contests through protected API endpoints.

1.3 Domain Vocabulary

Term	Definition
Challenge	A programming problem with a description, constraints, and a set of test cases.
Test case	A pair of input and expected output used to judge a submission.
Submission	A user's code attempt for a given challenge, stored with verdict and runtime.
Verdict	The outcome of a submission: Accepted, Wrong Answer, or Time Limit Exceeded.
Contest	A timed event featuring one or more challenges, with a live leaderboard.
Leaderboard	A ranked list of contest participants ordered by score and fastest solve time.
Hidden test case	A test case not shown to users; used to prevent hardcoded solutions.
Role	Either 'user' (competitor) or 'admin' (challenge/contest author).

2. Technology Stack

2.1 Server-Side

Technology	Role	Notes
Laravel 11	REST API framework	PHP 8.3, resource controllers, policies
Laravel Breeze	Auth scaffolding	Configured for API / SPA usage
Laravel Sanctum	Token-based auth	Stateless Bearer tokens for React SPA
SQLite	Database engine	Single-file DB, ideal for development & deployment
PHP artisan	CLI tooling	Migrations, seeders, tinker

2.2 Client-Side

Technology	Role	Notes
React 18	UI framework	Vite scaffold, JSX, functional components
React Router v6	Client-side routing	Protected routes, role-aware guards
TanStack Query v5	Server-state management	Caching, refetch intervals for leaderboard
@monaco-editor/react	Code editor	VS Code engine embedded in browser
Flowbite / Tailwind CSS	Component library & styling	Responsive design system
Axios	HTTP client	Intercepts to attach Bearer token automatically

3. Database Schema

3.1 Entity-Relationship Overview

The database consists of 6 tables with the following relationships:

- users 1 : N submissions
- users 1 : N contest_participants
- challenges 1 : N test_cases
- challenges 1 : N submissions
- contests N : M challenges (stored via contest_challenges pivot)
- contests 1 : N contest_participants

3.2 DBML (Database Markup Language)

```
Table users {
  id          integer [pk, increment]
  name        varchar
  email       varchar [unique]
  password    varchar
  role        varchar [default: 'user']  // 'user' | 'admin'
  score       integer [default: 0]
  created_at  timestamp
}

Table challenges {
  id          integer [pk, increment]
  title       varchar
  description text
  difficulty  varchar  // 'easy' | 'medium' | 'hard'
  time_limit  integer  // seconds
  created_by  integer [ref: > users.id]
  is_published boolean [default: false]
  created_at  timestamp
}

Table test_cases {
  id              integer [pk, increment]
  challenge_id    integer [ref: > challenges.id]
  input           text
  expected_output text
  is_hidden       boolean [default: false]
}

Table submissions {
  id           integer [pk, increment]
  user_id      integer [ref: > users.id]
  challenge_id integer [ref: > challenges.id]
  code         text
  language     varchar  // 'javascript' | 'python' | 'php'
  status       varchar  // 'accepted' | 'wrong_answer' | 'tle'
  score        integer [default: 0]
```

```
  runtime_ms   integer
  created_at   timestamp
}

Table contests {
  id          integer [pk, increment]
  title       varchar
  description text
  starts_at   timestamp
  ends_at     timestamp
  created_by  integer [ref: > users.id]
}

Table contest_challenges {  // pivot
  contest_id   integer [ref: > contests.id]
  challenge_id integer [ref: > challenges.id]
}

Table contest_participants {
  id           integer [pk, increment]
  contest_id   integer [ref: > contests.id]
  user_id      integer [ref: > users.id]
  total_score  integer [default: 0]
  rank         integer
  joined_at    timestamp
}
```

3.3 Table Descriptions

users

Stores all registered users. The role field controls access throughout the application — a user with role 'admin' has elevated permissions to create and manage challenges, test cases, and contests. The score field accumulates across all accepted submissions and is used for the global profile leaderboard.

challenges

Each row represents a programming problem. The created_by field links to the admin who authored it. is_published controls visibility: unpublished challenges are only visible to admins. The difficulty field ('easy', 'medium', 'hard') is used for filtering on the browse page. time_limit is stored in seconds and enforced server-side during submission evaluation.

test_cases

Belongs to a challenge (N:1). Each test case stores a raw input string and the exact expected output string. When a submission is evaluated, the server runs all test cases. Hidden test cases (is_hidden = true) are never returned to the client, preventing users from hardcoding outputs to pass visible cases only.

submissions

The most write-heavy table. Every time a user submits code, a row is inserted here. The status field records the verdict. score is computed at submission time: 100 points for Accepted, 0 otherwise. runtime_ms stores the milliseconds taken to evaluate all test cases, displayed on the profile page.

contest_challenges (pivot)

Implements the N:M relationship between contests and challenges. A contest can include multiple challenges; a challenge can appear in multiple contests. No extra columns — the relationship itself is the data.

contest_participants

Tracks which users have joined which contests. total_score and rank are recomputed every time a user submits during an active contest. This table is the source of truth for the live leaderboard.

4. API Routes Plan (REST)

All routes are prefixed with /api. Authenticated routes require a valid Sanctum Bearer token in the Authorization header. Owner-scoped routes additionally verify the authenticated user owns the resource.

4.1 Authentication

Method	Path	Function	Auth	Authorization
POST	/api/register	Register new user account	Public	Anyone
POST	/api/login	Authenticate and return Bearer token	Public	Anyone
POST	/api/logout	Revoke current Bearer token	Authenticated	Owner
GET	/api/user	Get current authenticated user info	Authenticated	Owner

4.2 Challenges

Method	Path	Function	Auth	Authorization
GET	/api/challenges	List all published challenges (filter by difficulty)	Public	Anyone
GET	/api/challenges/{id}	Get single challenge with visible test cases	Public	Anyone
POST	/api/challenges	Create a new challenge	Authenticated	Admin only
PATCH	/api/challenges/{id}	Update challenge details or publish status	Authenticated	Admin only
DELETE	/api/challenges/{id}	Delete a challenge and its test cases	Authenticated	Admin only

4.3 Test Cases

Method	Path	Function	Auth	Authorization
GET	/api/challenges/{id}/test-cases	List all test cases for a challenge (hidden ones only for admin)	Authenticated	Admin sees all; users see visible only

Method	Path	Function	Auth	Authorization
POST	/api/challenges/{id}/test-cases	Add a new test case to a challenge	Authenticated	Admin only
PATCH	/api/test-cases/{id}	Edit input, output, or visibility of a test case	Authenticated	Admin only
DELETE	/api/test-cases/{id}	Remove a test case	Authenticated	Admin only

4.4 Submissions

Method	Path	Function	Auth	Authorization
POST	/api/challenges/{id}/submissions	Submit code for a challenge; returns verdict	Authenticated	Owner (own submission)
GET	/api/submissions	Get all submissions by the current user	Authenticated	Owner only
GET	/api/submissions/{id}	Get single submission detail with test case results	Authenticated	Owner or Admin
DELETE	/api/submissions/{id}	Delete a submission record	Authenticated	Admin only

4.5 Contests

Method	Path	Function	Auth	Authorization
GET	/api/contests	List all upcoming and active contests	Public	Anyone
GET	/api/contests/{id}	Get contest details and challenges list	Public	Anyone
POST	/api/contests	Create a new contest	Authenticated	Admin only
PATCH	/api/contests/{id}	Update contest title, dates, or challenge list	Authenticated	Admin only
DELETE	/api/contests/{id}	Delete a contest	Authenticated	Admin only
POST	/api/contests/{id}/join	Join a contest as a participant	Authenticated	Any user
GET	/api/contests/{id}/leaderboard	Get live-ranked participants for a contest	Public	Anyone

4.6 User Profile

Method	Path	Function	Auth	Authorization
GET	/api/profile	Get own profile, stats, and submission history	Authenticated	Owner
PATCH	/api/profile	Update own name or password	Authenticated	Owner
GET	/api/users	List all users with scores (admin panel)	Authenticated	Admin only
PATCH	/api/users/{id}/role	Promote or demote a user's role	Authenticated	Admin only

5. Client-Side Application Design

5.1 Non-Trivial Client-Side Feature: Monaco Code Editor with Live Verdict Panel

Feature type In-browser interactive code editor with dynamically changing output state

The primary client-side feature is a split-pane code editing environment on the Challenge Detail page. It is built using the @monaco-editor/react library, which embeds the same editor engine that powers Visual Studio Code directly in the browser.

This feature is non-trivial because it involves complex interdependent state: the selected programming language changes the Monaco syntax highlighting mode; the submission status drives the entire verdict panel's content and color; and test case results arrive as an array and must be mapped to individual pass/fail indicators dynamically. All of this happens without a page reload.

Key interactive behaviors

- Language selector dropdown — switching language (JavaScript / Python / PHP) re-initializes Monaco with the correct grammar and updates the code template shown in the editor.
- Submit button — triggers a POST request via TanStack Query mutation. While pending, the button shows a spinner and is disabled.
- Verdict panel — dynamically renders based on submission status: green 'Accepted' banner, red 'Wrong Answer' with diff output, or orange 'Time Limit Exceeded' message.
- Test case tabs — visible test cases render as tabs. Each tab shows input, expected output, and actual output. Tabs are highlighted green or red based on per-case pass/fail.

5.2 Non-Trivial Client-Side Feature: Real-Time Contest Leaderboard

Feature type Live-updating ranked list with countdown timer and animated rank changes

The Contest Detail page hosts a leaderboard that automatically refreshes every 5 seconds during an active contest using TanStack Query's refetchInterval option. A countdown timer (built from the contest's ends_at timestamp) ticks every second in the browser. As the leaderboard data refreshes, rank changes are animated using CSS transitions so users can see when they move up or down.

5.3 Non-Trivial Client-Side Feature: Multi-Step Challenge Builder (Admin)

Feature type Multi-level dynamically changing form — fields and validation change at each step

Admin users access a 4-step challenge creation wizard. Each step is a distinct form section with its own validation rules that must pass before the user can proceed to the next step. The form state is held in React state across all steps and only submitted as a single API call on final confirmation.

- Step 1 — Metadata: title, description, difficulty selector, time limit. Difficulty selection changes the time limit suggestions shown.
- Step 2 — Test cases: a dynamic list where admins add, remove, and edit input/output pairs. A toggle marks each as hidden or visible.
- Step 3 — Constraints: a free-text constraints editor and an optional example code snippet field.
- Step 4 — Preview & publish: shows a read-only preview of the challenge as a user would see it, with a final Publish toggle.

6. Functional Requirements

6.1 Roles

Role	Description
Guest	Unauthenticated visitor. Can browse published challenges and view contest listings. Cannot submit code or join contests.
User	Registered and authenticated. Can solve challenges, submit code, join contests, and manage their own profile.
Admin	Elevated role. Can create, edit, and delete challenges, test cases, and contests. Can view all submissions and manage user roles.

6.2 Features by Role

Guest

- Browse all published challenges with difficulty filter
- View challenge description and visible test cases
- View upcoming and active contests
- View global leaderboard for ended contests
- Register for a new account
- Log in

User (all Guest features, plus)

- Submit code solutions for any published challenge
- View own submission history and verdicts
- Join an active contest
- View live leaderboard during an active contest
- View own profile with stats: total submissions, accepted count, global score
- Update own display name and password

Admin (all User features, plus)

- Create, edit, and delete challenges
- Add, edit, and delete test cases (including hidden ones)
- Publish or unpublish challenges
- Create, edit, and delete contests
- Add or remove challenges from a contest
- View all submissions from any user
- Promote or demote other users' roles

6.3 Features by Role and Page

Page	Guest	User	Admin
Challenge List	Browse, filter by difficulty	Browse + see own solve status	Browse + see unpublished challenges
Challenge Detail	Read description, see examples	Read + submit code + see verdict	Read + submit + edit/delete challenge
Contest List	View all contests	View + join contest	View + create contest
Contest Detail	View leaderboard (if ended)	View leaderboard (live) + submit	View + manage challenges in contest
Profile Page	Not accessible	Own stats, history, edit details	Own stats + user management panel
Challenge Builder	Not accessible	Not accessible	Full 4-step wizard
Submission Detail	Not accessible	Own submissions only	Any user's submissions

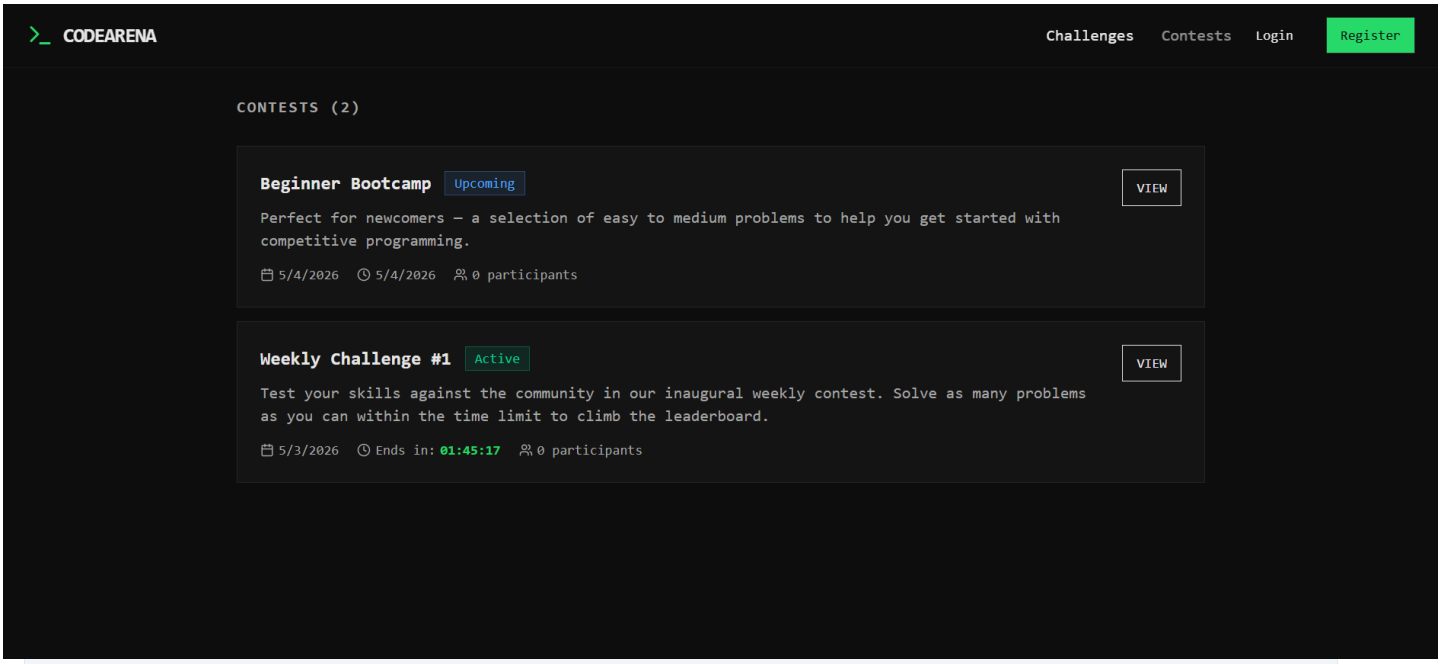
6.4 Permissions / Authorization Matrix

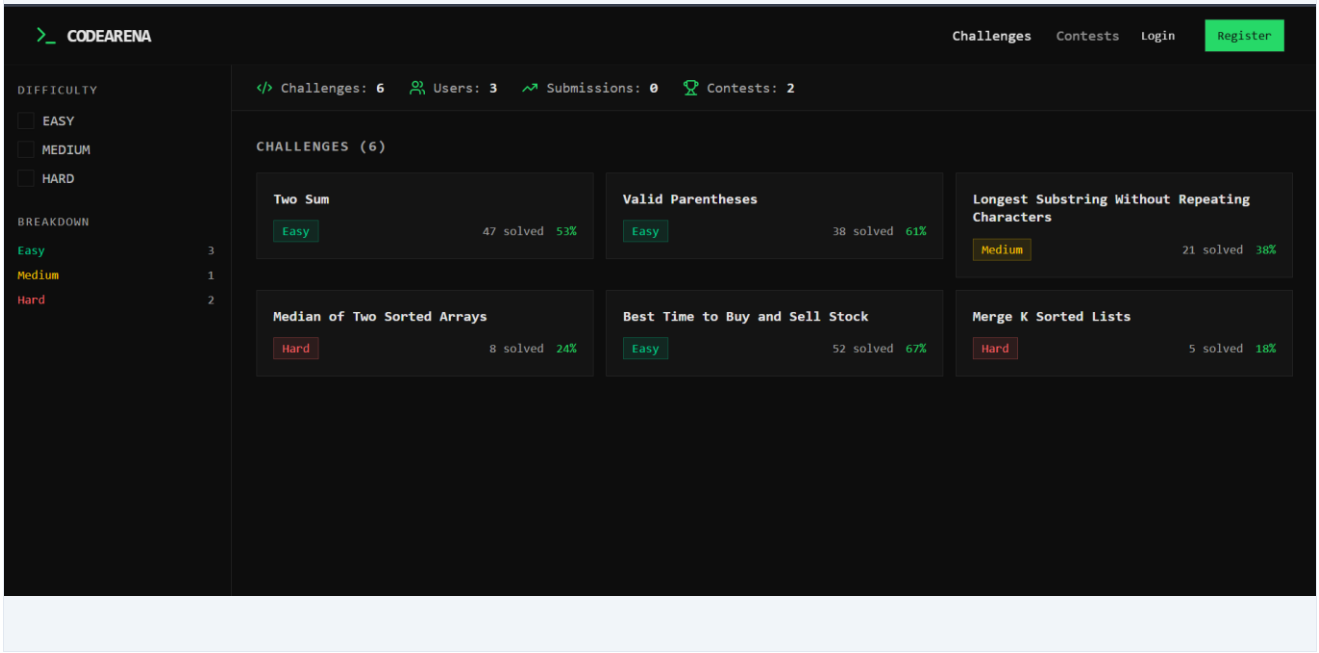
Action	Guest	User	Admin	Scope rule
View published challenges	Yes	Yes	Yes	Public
View unpublished challenges	No	No	Yes	Admin only
Create challenge	No	No	Yes	Admin only
Edit / delete challenge	No	No	Yes	Admin only
View visible test cases	Yes	Yes	Yes	Public
View hidden test cases	No	No	Yes	Admin only
Submit code	No	Yes	Yes	Authenticated
View own submissions	No	Yes	Yes	Owner only
View any submission	No	No	Yes	Admin only
Join contest	No	Yes	Yes	Authenticated
Create / manage contest	No	No	Yes	Admin only
View contest leaderboard	Yes (ended)	Yes (live)	Yes (live)	Public (ended), Auth (live)
Edit own profile	No	Yes	Yes	Owner only
Manage user roles	No	No	Yes	Admin only

7. UI Mockup Descriptions

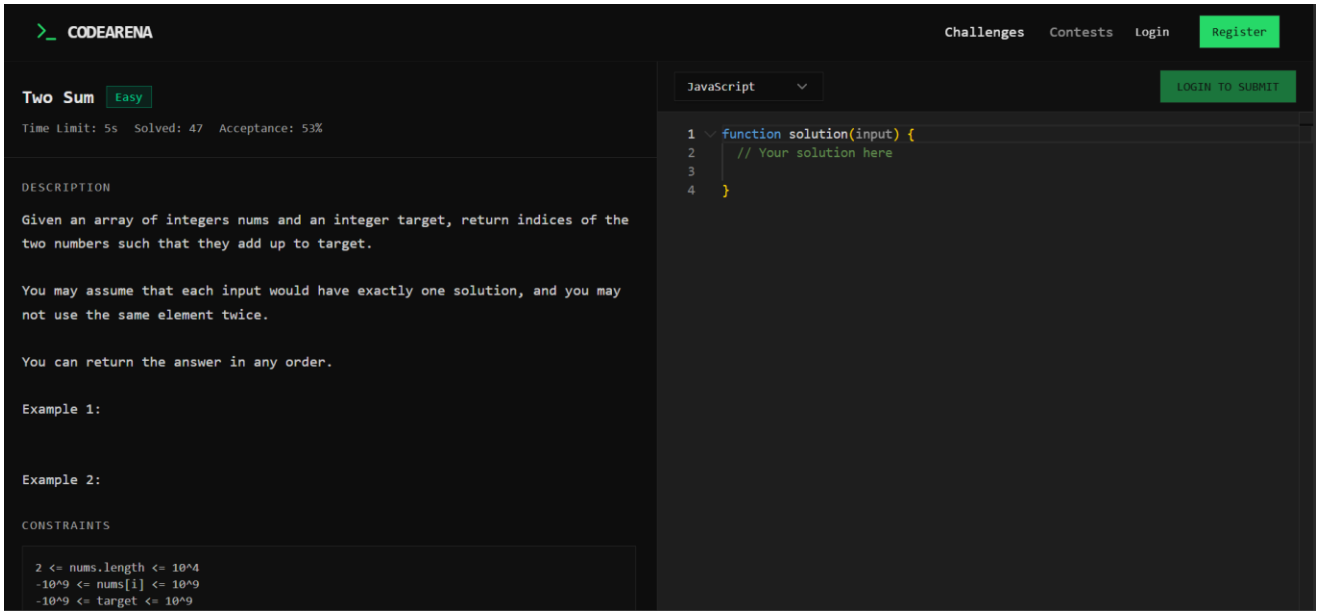
The following describes each major screen's layout and key components. The application uses Tailwind CSS with Flowbite components for a consistent, responsive design system.

7.1 Page: Challenge List (/)



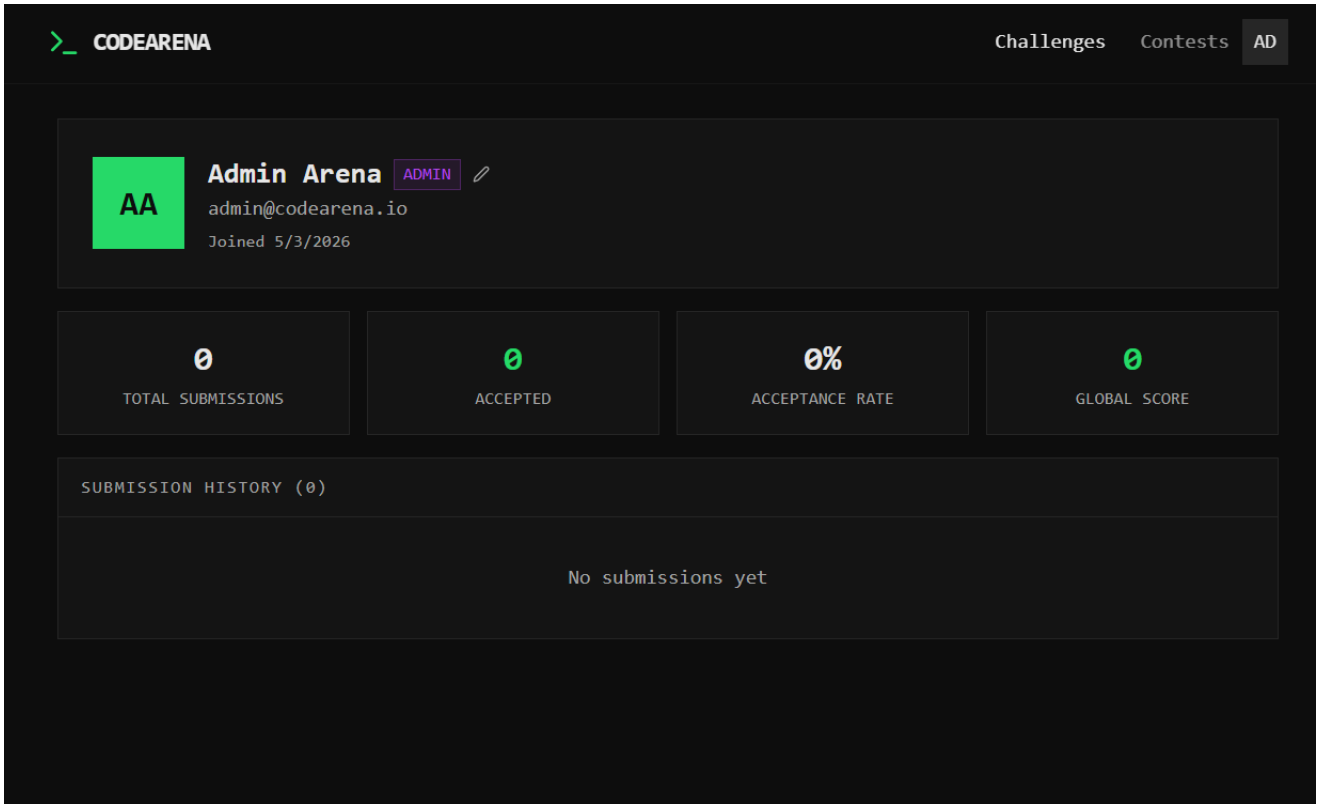


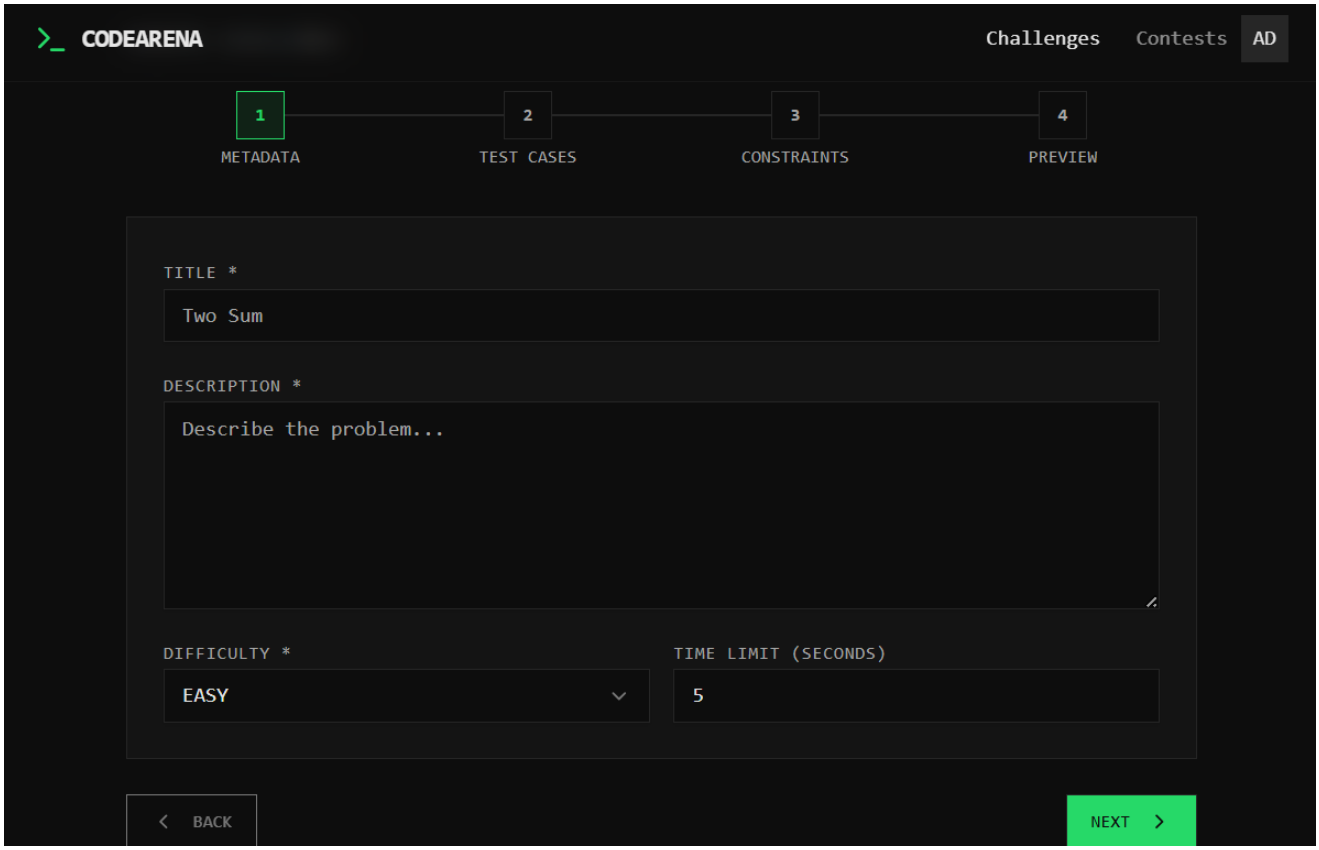
7.2 Page: Challenge Detail (/challenges/:id)



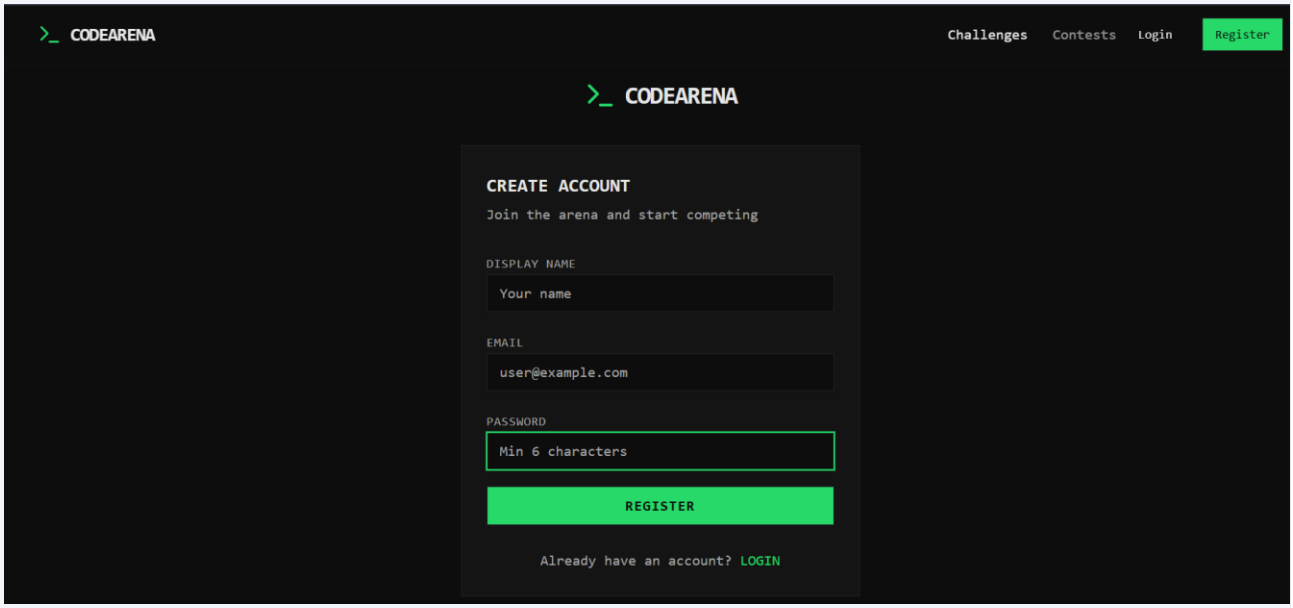
7.3 Page: Contest Detail (/contests/:id)

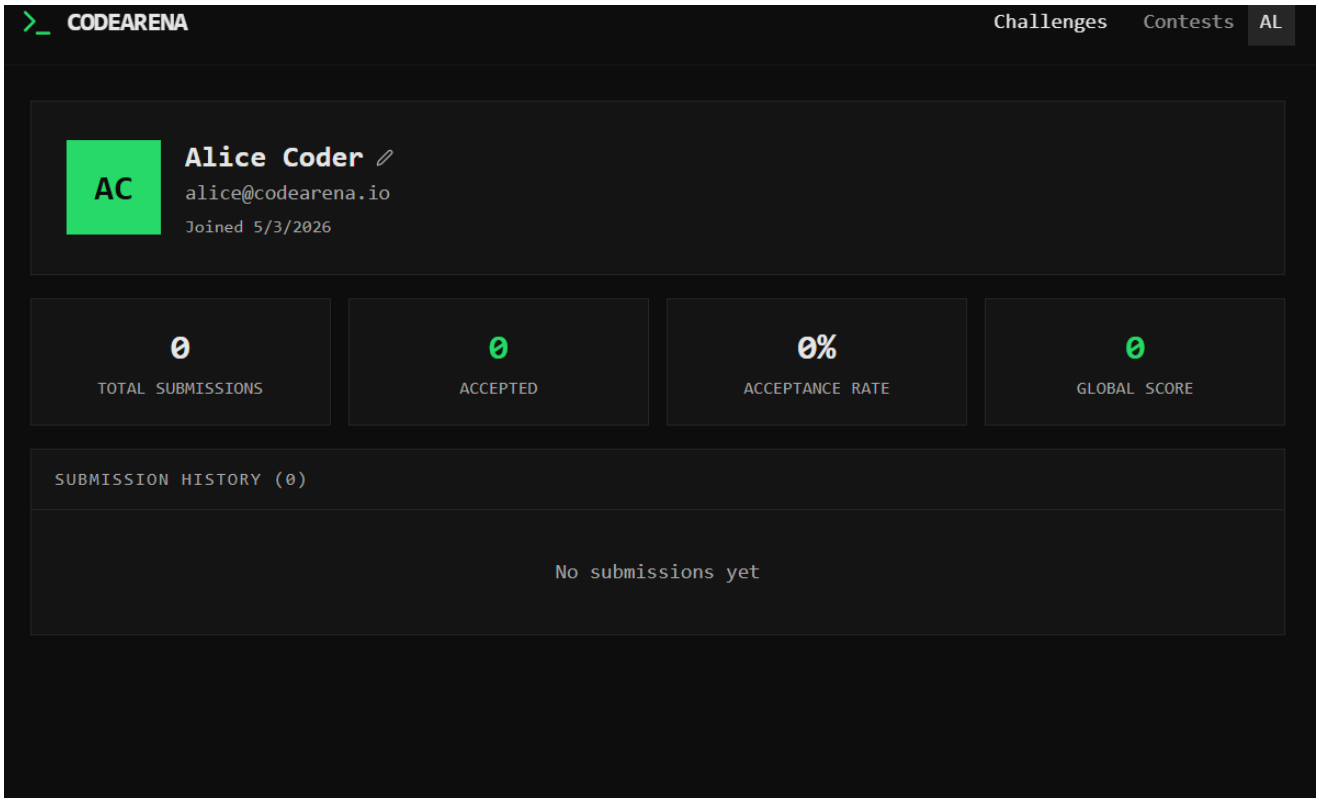
7.4 Page: Challenge Builder (/admin/challenges/create)


CODEARENA
Challenges
Contests
AD
AA
Admin Arena
ADMIN
admin@codearena.io
Joined 5/3/2026
0
TOTAL SUBMISSIONS
0
ACCEPTED
0%
ACCEPTANCE RATE
0
GLOBAL SCORE
SUBMISSION HISTORY (0)
No submissions yet


CODEARENA
Challenges
Contests
AD
1
METADATA
2
TEST CASES
3
CONSTRAINTS
4
PREVIEW
TITLE *
Two Sum
DESCRIPTION *
Describe the problem...
DIFFICULTY *
EASY
TIME LIMIT (SECONDS)
5
BACK
NEXT

7.5 Page: User Profile (/profile)





8. AI Appendix

8.1 Use of AI Tools

Artificial intelligence tools were used during the design and documentation phase of this project. The following describes the nature of that use:

Tool used	Claude (Anthropic) — conversational AI assistant

How AI was used

- Brainstorming the project concept — AI was used to evaluate project ideas against the assignment's requirements and suggest which would be most technically substantive.
- Structuring the documentation — AI suggested the recommended sections and order for the Phase 1 document based on the assignment specification.
- Drafting initial descriptions — Rough descriptions for the database schema, API routes, and functional requirements were drafted with AI assistance and then reviewed and adjusted by the author(ME (Maghan Das).
- DBML schema review — The AI reviewed the initial entity design for normalization issues and suggested adding the contest_challenges pivot table for the N:M relationship.

What AI was not used for

- No code was generated by AI for this phase — the documentation is a design artifact only.
- All final decisions about project scope, entity design, and feature selection were made by me.